

Asynchronous programming

concurrency on one thread

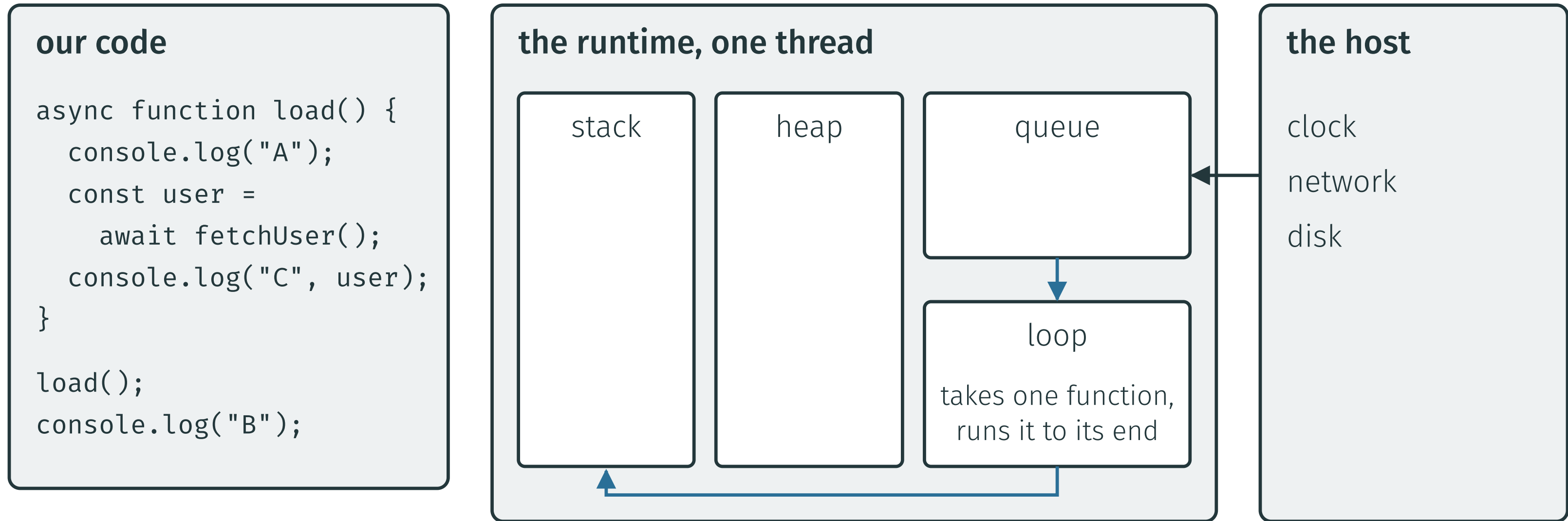
What does a program do while it waits, and where is the rest of a function while it waits?

The model and its properties

one thread, four properties

The model

one thread, a stack, a heap, a queue, and a host



Two assumptions hold for the whole block.

One thread: all our code runs on it; no other thread runs code of ours.

An uncontrolled environment: the order of the host's events is not the program's to fix.

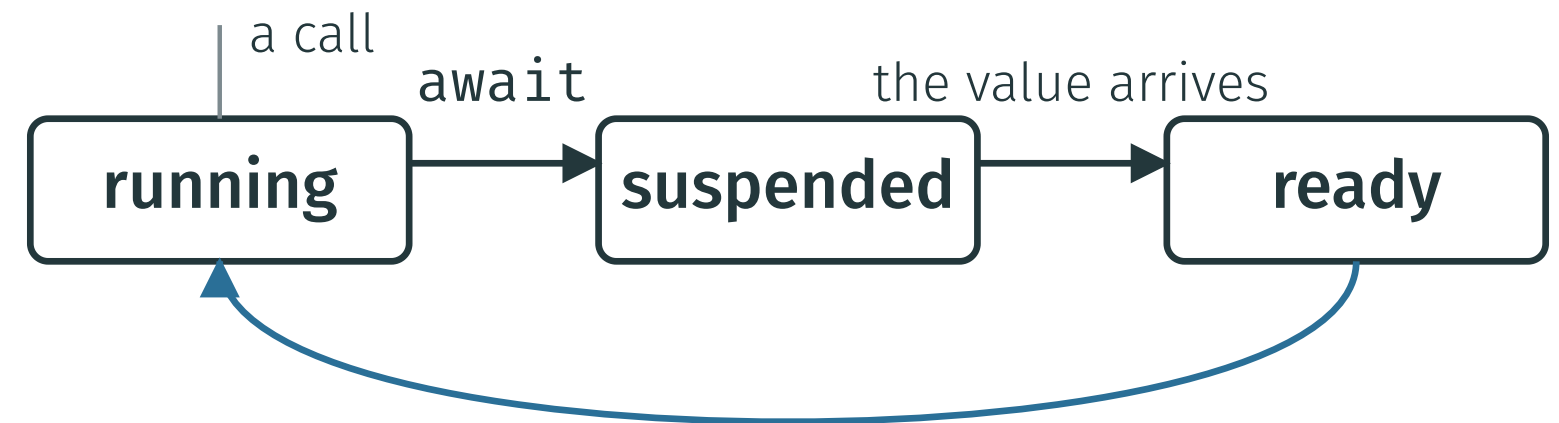
Suspension points

cooperative transfer of control

JAVASCRIPT

```
async function load() {  
  console.log("A");  
  const user = await fetchUser();  
  console.log("C", user);  
}
```

```
load();  
console.log("B");
```



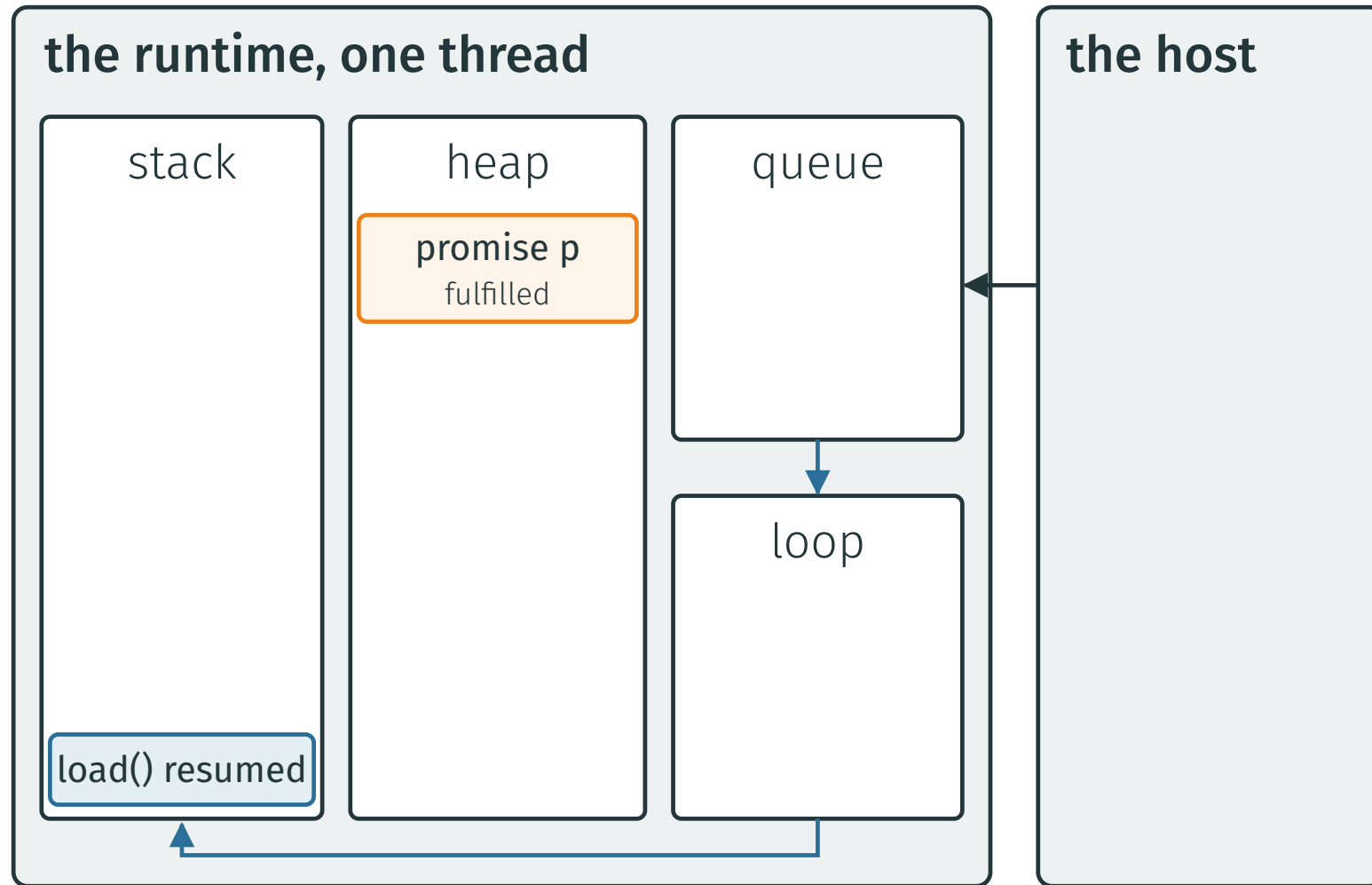
the loop takes it, when the stack is empty

The rest, the highlighted line, is kept, registered on the awaited **promise**: a value that stands for a result to come.

Definition. An **asynchronous function** can be **suspended** at the points marked `await` in its text. At a **suspension point** its frame leaves the stack; **the part that remains to be run**, with its local variables, is kept in the heap. The caller resumes.

Property 1 · Suspension. An asynchronous function is suspended only at an `await` written in it. Its caller then goes on with its next statement: after `load();` the script prints B. A caller that writes `await load();` is suspended too.

Creating and awaiting a promise [†]`new Promise` and `await`



The loop runs the rest: `load()` resumes, and the `await` expression has `p`'s value.

`await e`: waiting and resuming

```
const user = await fetchUser();
```

JAVASCRIPT

1. It evaluates `e`. If the result is not a promise, it is wrapped into an already fulfilled one.
2. It registers the rest of the function on that promise, as one of its waiters: `await p` amounts to `p.then(rest)`^{*}.
3. It suspends the function: its frame leaves the stack, and its caller goes on.
4. When the promise is fulfilled, the rest goes to the microtask queue; the loop runs it, and `await e` has the promise's value.

[†] Python, C# and Rust have the same pair; what differs around it: slides 9 to 11, and the appendix.

^{*} `p.then(f)` registers `f`, to be called with the value of `p` once `p` is fulfilled.

Run-to-completion

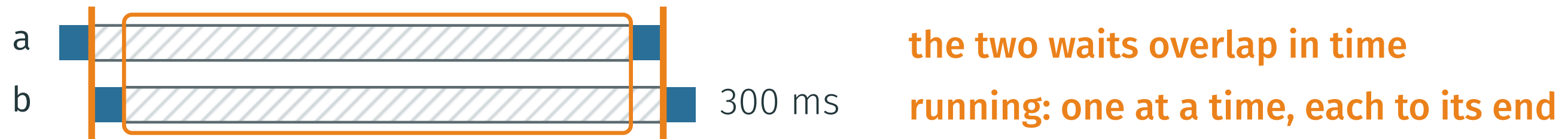
the loop's rule, and two consequences

a and **b**: two requests, each answered after 300 ms; made in turn, or together before either is awaited.

in turn



together



Property 2 • Waiting is not running. Waits can overlap in time; executions of our code cannot: on one thread*, at any moment at most one computation is running.

Property 3 • Run-to-completion. The loop takes the next function from the queue only when the stack is empty: the running code has returned, and so has everything it called. Running code is never interrupted: it keeps the thread until it returns.

* JavaScript can run code of ours on another thread, a **worker**, with its own loop and its own heap.

Interleavings and atomicity

where one computation can run inside another

P's value arrives first



a suspension point:
the only place another
computation can run

Q's value arrives first



a stretch with no await:
atomic for every other
computation

■ running ▨ waiting

Execution: one sequence of resumptions. One program and one input have as many executions as the environment can produce.

Interleaving points are exactly the **suspension points**. Between two of them, nobody else runs.

Property 4 · Atomicity between suspension points. A stretch of a function's text with no `await` in it is **atomic** with respect to all other code of ours: nobody observes it half done, except the functions it calls, which run inside it. Others see all of it or none of it, and they can run between two stretches.

The race on one thread

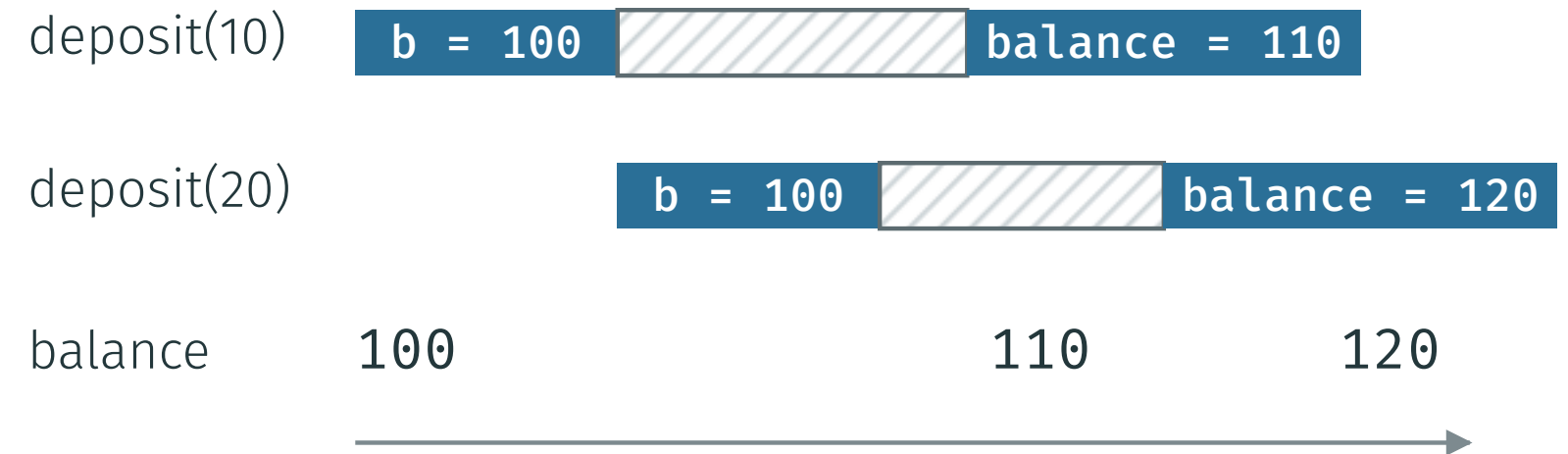
a read, a suspension point, a write

```
JAVASCRIPT
let balance = 100;
async function deposit(n) {
  const b = balance;
  await approve(n);
  balance = b + n;
}
deposit(10);
deposit(20); // required: 130
```

`approve(n)` asks another service to approve the deposit; its answer comes 100 ms later.

Race condition: the outcome depends on an order the program does not fix. Here an **atomicity violation:** the deposit is split across a suspension point.

A **linearization point** of a call: an instant between the call and its return at which it takes effect. No choice of one for each deposit gives 120.



A lock? A thread's lock cannot help: there is one thread, and holding it across the wait blocks it.
An asynchronous lock, acquired with `await`, can.

Starting the work at the call, or later

Properties 1 to 4 describe JavaScript. Three questions, each decided by a small run, locate any other language with respect to them.

First: when does an asynchronous function start to run? At the call, or only when a scheduler takes it? The run: `load()`, then `console.log("B")`.

language	a call
JavaScript, C#	runs it up to its first <code>await</code> : A, B, C Ada
Python	a coroutine object, nothing runs: a warning, then B; <code>create_task</code> : B, A, C Ada
Rust	builds a future, which runs when polled
Kotlin	a <code>suspend</code> call waits: A, C Ada, B; <code>launch</code> queues it
Go, Java	a plain call waits: A, C Ada, B; with <code>go</code> or a virtual thread: B, A, C Ada in these runs, A and B in no fixed order

Two realizations of one model. In JavaScript and C# the call runs the function to its first `await`. In Python and Rust the call builds an object and runs nothing, session 5's thunk usable once: `await` runs it in place, `create_task` or `spawn` queues it.

The suspension point

at every `await`, or only when the value is missing

Second: what does an `await` do when the value is already there? Suspend anyway, so that every `await` is an interleaving point; or go straight on, so that the stretch stays atomic.

The run: the two deposits, with an `approve` that has nothing to wait for. In JavaScript, `await null` awaits a value already there; in Python, `asyncio.sleep(0)` asks for a wait of zero seconds.

language	an <code>await</code> on a finished result
JavaScript	suspends, even <code>await null</code> : 120
Python	goes straight on: 130; <code>await sleep(0)</code> suspends: 120
C#, Rust, Kotlin	goes straight on
Go, Java	there is no <code>await</code>

Of these languages only JavaScript suspends at every `await`. Elsewhere an `await` on a value already there is not a suspension point: on one thread, the stretch around it stays atomic.

The resuming thread

one thread, or a pool

Third: which thread resumes a suspended computation? The one thread of the loop, as the model assumes; or any thread of a pool.

The run: the two deposits, with a real wait, on the runtime each language gives by default.

language	the thread that resumes
JavaScript	the one thread: always 120
Python	the loop's one thread: always 120
C#	in a console program, a thread of a pool: 110 or 120 in these runs
Rust, Kotlin	the executor's or the dispatcher's choice
Go, Java	several threads: 110 or 120 in these runs; 130 possible

Where a pool resumes the computations, two of them run at the same time: the model's first assumption is gone, and Property 4 with it, although every suspension point is written. Session 7.

Worked examples

the programs of part 1, run on the stage

1. One `await`
2. Two requests of 300 ms
3. A timer and a computation
4. Two deposits

Each one is run in the demo, step by step, on the model's parts: our code, the runtime, the host.

Example 1: one await

```
const fetchUser = () =>
  new Promise(res =>
    setTimeout(() => res("Ada"), 300));

async function load() {
  console.log("A");
  const user = await fetchUser();
  console.log("C", user);
}
await load();
console.log("B");
```

JavaScript ↗

Run

```
A
C Ada
B
```

`fetchUser` answers with `"Ada"` after 300 ms: a request to a server, imitated by a timer, which calls `res("Ada")` and so fulfills the promise.

What is printed, in which order?

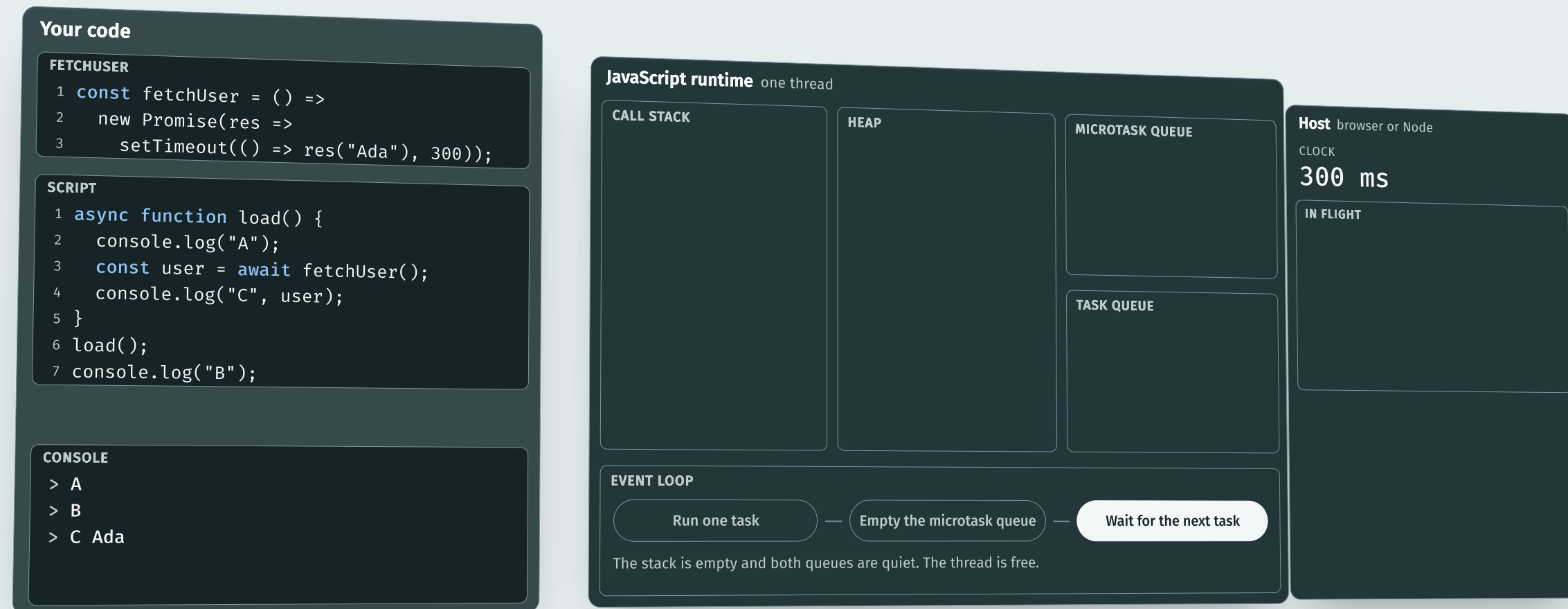
1. A, B, C Ada
2. A, C Ada, B
3. B, A, C Ada

With `await load();` instead, the script is suspended too, and prints B last: A, C Ada, B.

Demo 1: the run of `load` step by step, on the model's four parts

`load` prints A, awaits a user that arrives after 300 ms, and prints C with the name. The script prints B.

Property 1: suspended only at the `await`; the caller continues. A, B, C Ada.



An `await` is this round trip: off the stack, into the heap, through the queue, back on the stack.



Example 2: two requests of 300 ms

```
async function inTurn() {  
  const a = await get("a");  
  const b = await get("b");  
  return [a, b];  
}  
  
async function together() {  
  const pa = get("a"), pb = get("b");  
  return [await pa, await pb];  
}  
  
let t = Date.now();  
console.log(await inTurn(), Date.now() - t);  
t = Date.now();  
console.log(await together(), Date.now() - t);
```

JavaScript ↗

`get` answers after 300 ms. In `together`, both requests have left before either is awaited.

How long does each function take?

1. 300 ms each
2. `inTurn` 600 ms, `together` 300 ms
3. 600 ms each

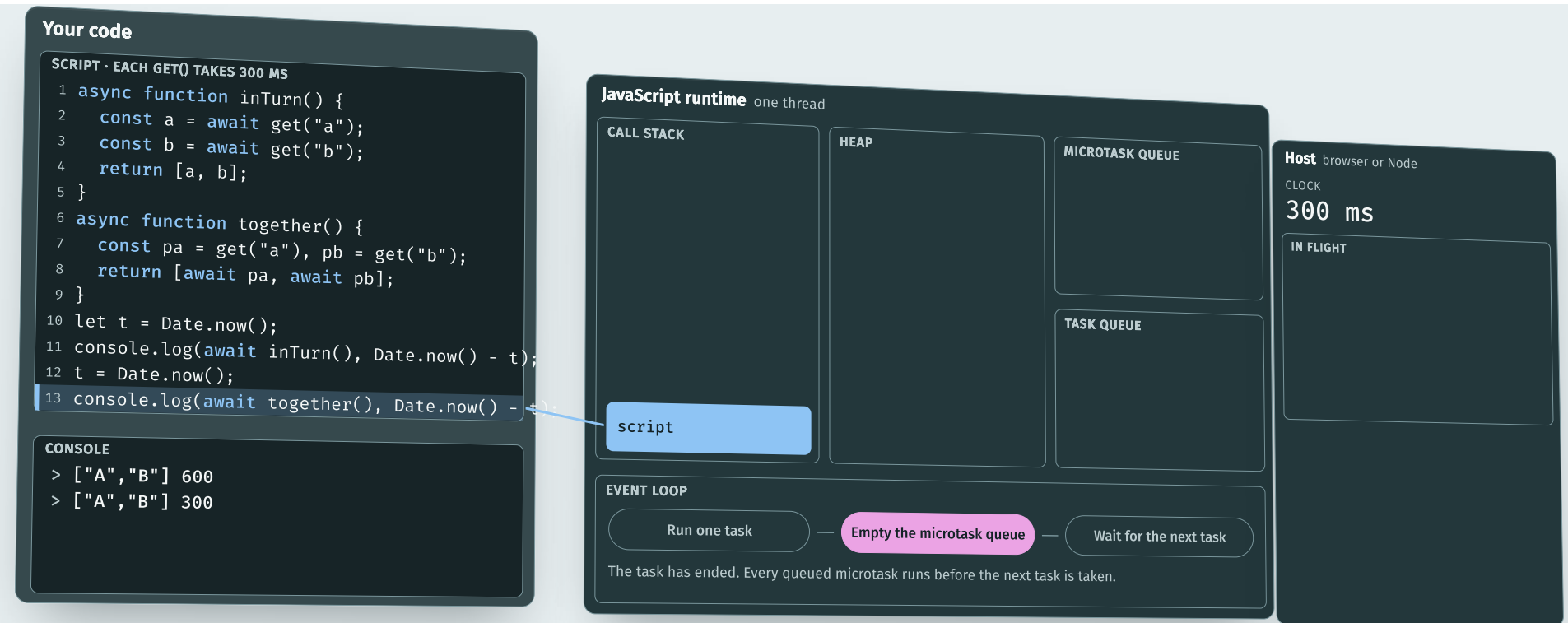
Run

```
["A", "B"] 600  
["A", "B"] 300
```

The text decides: a request written after an `await` leaves after it. Start the work first, then await.

Demo 2: two requests in turn, then together

Two values, a and b, each 300 ms away. inTurn awaits a, then asks for b; together asks for both first.



Property 2: the waits overlap, the running stretches never do. 600 against 300 ms.



The script prints 300 ms. The difference is where the calls stand relative to the await: start the work first, await afterwards.

Example 3: a timer and a computation

```
const t0 = Date.now();
setTimeout(() => console.log("tick at",
    Date.now() - t0, "ms"), 100);
// computes for 500 ms, never awaits
while (Date.now() - t0 < 500) {}
console.log("done at",
    Date.now() - t0, "ms");
```

JavaScript ↗

Run

recorded · node 25

```
done at 500 ms
tick at 504 ms
```

A timer due at 100 ms, then a loop that computes for 500 ms without reaching a suspension point.

When does `tick` print?

1. at 100 ms
2. at about 500 ms
3. never

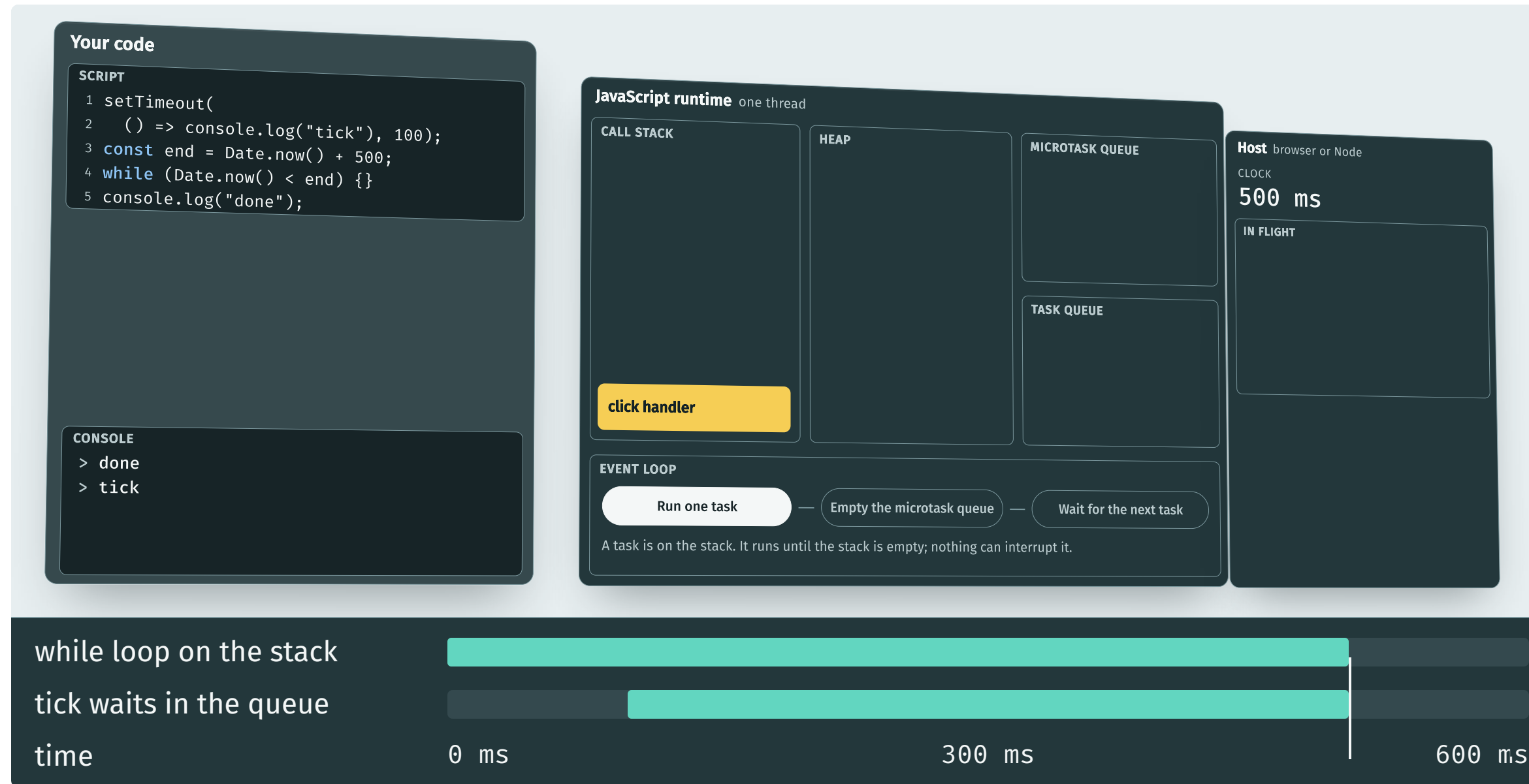
Property 3: the timer's function is in the queue from 100 ms; the loop takes it only when the running code has returned. `done` at 500 ms, then `tick`.

Keeping the thread free is our job: split the computation, or move it to another thread, a worker.

Demo 3: a timer and a computation

A timer of 100 ms, then a loop that computes for 500 ms without an await.

Property 3: running code keeps the thread until it returns. `done` at 500 ms, then `tick`.



Cooperative: the loop runs the next function only where our code gives the thread back. Long work is split up, or moved to another thread.

Example 4: two deposits

```
let balance = 100;
async function deposit(n) {
  const b = balance;
  await approve(n);
  balance = b + n;
}
deposit(10);
deposit(20);
```

JavaScript ↗

Run

Run, first approval slower

balance 110

`approve` answers after 100 ms. Required: 130 when both calls have finished.

What is the balance then?

1. 130
2. 120
3. 110

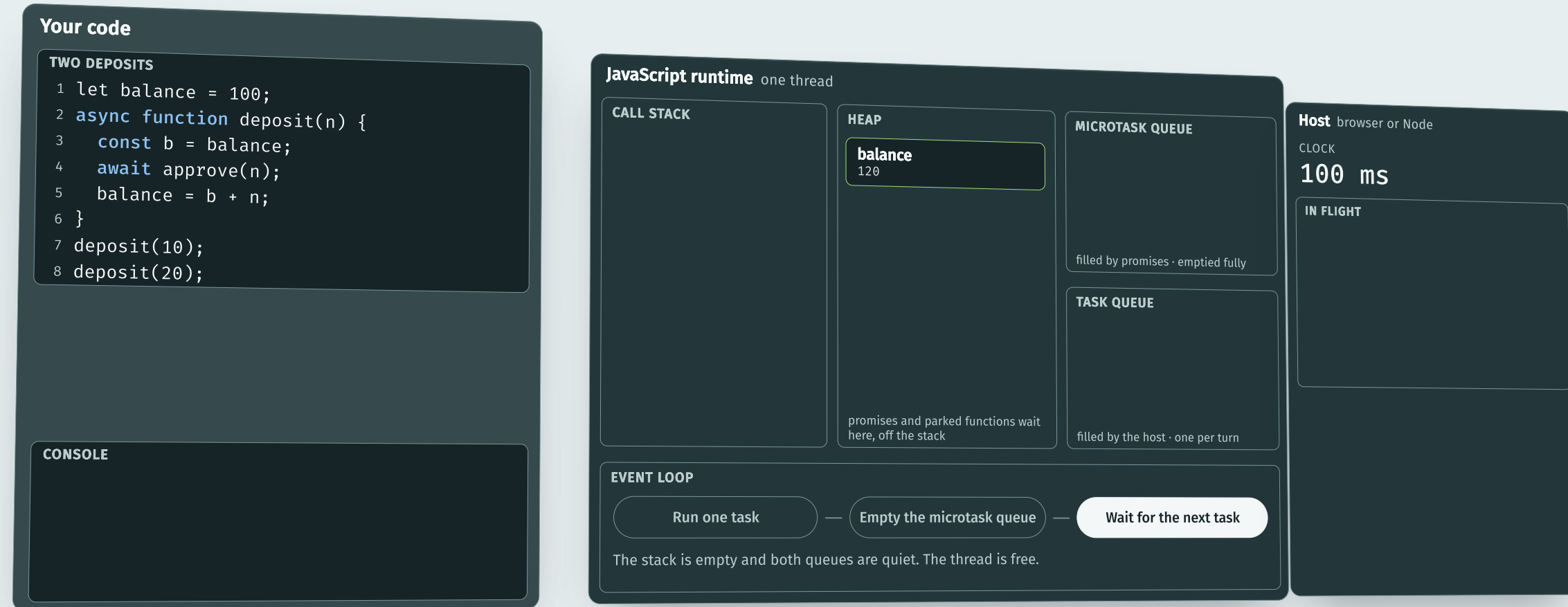
It happens wherever we read shared state, wait, then write it:
a counter updated after a request, a cache refreshed after a fetch.

Demo 4: two deposits

equal approvals; then the first one slower

Two deposits on one balance, 10 and 20, each suspended at its approval. Required: 130.

Property 4 ends at the **await**: the read and the write are two stretches, and the other call runs between them. 120.



120, where 130 was required: each call was suspended between its read and its write, and the other ran there. First approval slower: 110.

The same programs in Python

a bare call; an `await` on a finished result

```
async def main():
    load()                # the call alone
    print("B")
    await asyncio.sleep(0.5)
    asyncio.create_task(load()) # queued
    print("B")
    await asyncio.sleep(0.5)
```

Python ↗

Run

```
<exec>:10: RuntimeWarning: coroutine 'load' was never awaited
RuntimeWarning: Enable tracemalloc to get the object allocation
B
B
A
C Ada
```

Does a call start the work?

A coroutine object that is never awaited never runs:
its work is lost, and the warning is the only sign of it.

```
async def trial(name, approve):
    account = {"balance": 100}
    async def deposit(n):
        b = account["balance"]
        await approve()
        account["balance"] = b + n
    await asyncio.gather(
        deposit(10), deposit(20))
    print(name, account["balance"])
```

Python ↗

Run

```
await trial("nothing to wait for:", done)
await trial("waits 0 seconds:", sleep0)
```

```
nothing to wait for: 130
waits 0 seconds: 120
```

Is every `await` a suspension point?

Conclusion

the four properties; what is left open; the lab, and session 7

Conclusion of block 2

- 1 Suspended only at an `await` in it; after `load()`; the caller goes on, after `await load()`; it waits.
- 2 Waits overlap; on one thread, running never does.
- 3 Run-to-completion: running code keeps the thread until it returns.
- 4 A stretch with no `await` is atomic. The race on one thread is an atomicity violation, not a data race.

the language guarantees

atomicity between suspension points, given one thread

a library gives

combinators over promises:
`Promise.all`, `gather`

the programmer keeps

every invariant that must survive a wait

Left open: what the kept rest of a function is; who holds it while the function waits; who invokes it, and when. The lab's optional quest.

Next

the lab, and session 7

- lab** The warm-up: the two blocks of today, on JavaScript with Python beside it. The optional quest: a small page on continuations, with exercises on paper; the appendix of this deck is its reading.
-
- 7** Several threads on one memory. The repaired deposit, run by two Java threads twenty thousand times: 130 in about 98 runs of 100, 110 or 120 in the others. The suspension points are gone, and with them the atomic stretch.

The suspended computation as a value

continuations, a promise, a loop, a generator and a driver

Continuations

the kept rest, written by hand

```
function load() {  
  console.log("A");  
  return fetchUser().then(user => { console.log("C", user); });  
}  
load();  
console.log("B");
```

JavaScript ↗

Run

A
B
C Ada

Definition. The **continuation** of a point in a computation is the rest of the computation from that point, as a function of the value expected there.

In `double(x) + 1`, while `double(x)` is computed, the rest is the context `[·] + 1`: the function `v => v + 1`. At the `await` of `load`, the rest is `user => { console.log("C", user); }`.

Continuation-passing style

a function that cannot return takes its continuation

JAVASCRIPT

```
// doubleK asks a server for 2x; the answer goes to k, 100 ms later  
const doubleK = (x, k) => setTimeout(() => k(2 * x), 100);  
const f = x => double(x) + 1; // double cannot return: write fK
```

```
const fK = (x, k) => doubleK(x, v => k(v + 1)); // A  
const fK = (x, k) => doubleK(x, k) + 1;        // B  
const fK = (x, k) => k(doubleK(x) + 1);        // C
```

JavaScript ↗

Run

```
recorded · node 25  A gives k the value 7  
B gives k the value 6  
C gives k the value 11, then fails: k is not a function
```

The cost: every function that needs the result of `fK` for its own result takes a continuation too, up the chain of calls. Hence `async` on every function that uses `await`, and on every caller that awaits its result.

A promise by hand

a write-once cell that holds continuations

```
class P {  
  state = "pending"; value; waiters = [];  
  then(k) {  
    if (this.state === "pending") this.waiters.push(k);  
    else later(() => k(this.value));  
  }  
  resolve(v) {  
    if (this.state !== "pending") return;  
    this.state = "fulfilled"; this.value = v;  
    for (const k of this.waiters) later(() => k(v));  
  }  
}
```

JavaScript ↗

A promise nobody resolves: its waiters never run.

If nothing can reach it any more, the collector frees it with them, silently. If something still can, they stay for ever: a memory leak.

`resolve` has the waiting continuations. Call each of them now, inside `resolve`, or put each in the queue for the loop?

The queue and the loop the scheduler

JavaScript ↗

```
const ready = []; let scheduled = false;
function later(k) { // put k in the queue
  ready.push(k);
  if (!scheduled) { scheduled = true; setTimeout(run, 0); }
}
function run() { // the loop: one at a time, to its end
  scheduled = false;
  while (ready.length > 0) { const k = ready.shift(); k(); }
}
```

Run the test, both ways

recorded · node 25

```
call now   : k 1 | after resolve
queue      : after resolve | k 1
```

JavaScript queues: a function given to `then` is never called before the current code has run to completion (Promises/A+, 2.2.4). Property 3 is these lines. Property 4 is kept by `then` and `resolve` calling `later` instead of `k`.

A value sent into a generator

session 5's object, resumed with a value

```
function* two() {  
  const x = yield 1;  
  console.log("x is", x);  
}  
const g = two();  
console.log(g.next());  
console.log(g.next("Ada"));
```

JavaScript ↗

Run

```
{ value: 1, done: false }  
x is Ada  
{ value: undefined, done: true }
```

`g.next(v)` makes `v` the value of the `yield` where `g` was suspended.

```
function* loads() {  
  for (let i = 0; i < 3; i++) {  
    const user = yield fetchUser();  
    console.log(i, user);  
  }  
}  
drive(loads);  
console.log("B");
```

JavaScript ↗

Run

```
B  
0 Ada  
1 Ada  
2 Ada
```

A wait inside a loop: the generator's object keeps `i` and the position.

An async function as a generator and a driver

await became yield

```
function drive(gen) {
  const g = gen();
  const step = v => {
    const r = g.next(v);
    if (!r.done) r.value.then(step);
  };
  step();
}
```

JavaScript ↗

Run

A
B
C Ada

A, B, C Ada on our promise, our queue, our loop and our driver: what the language did.

code

VE

```
function drive(gen) {
  const g = gen();
  const step = v => {
    const r = g.next(v);
    if (!r.done) r.value.then(step);
  };
  step();
}
```

IPT

```
function* load() {
  console.log("A");
  const user = yield fetchUser();
  console.log("C", user);
}
```

SOLE

A

JavaScript runtime one thread

CALL STACK

HEAP

p, a P
fulfilled: "Ada"

g, a generator
position 2 · us

promises and parked f
here, off the stack

step("Ada") VIA PROMISE
calls g.next("Ada")

EVENT LOOP

run, our loop, takes step from ready and calls it. g.next("Ada") puts Ada where the yield was, and load prints C Ada.

Languages against the model

task management and stack management

language	a call	an <code>await</code> on a finished result	the thread that resumes	who keeps the suspended state
JavaScript	runs to the first <code>await</code>	suspends	the one thread	the compiler (stackless)
C#		goes straight on	a thread of a pool (console)	
Python (asyncio)			the loop's one thread	
Rust			the executor's choice	
Kotlin	a <code>suspend</code> call waits; <code>launch</code> queues		the dispatcher's choice	
Go, Java (virtual threads)	a call waits; <code>go</code> or a virtual thread starts it apart	no <code>await</code>	several threads	the runtime (stackful)
OCaml 5 (effects)	runs until it performs an effect		one domain; a scheduler in a library (not run)	

Task management: who may run between two of our lines. **Stack management:** who keeps the suspended state. Two independent questions (Adya et al., 2002).

Worker threads

another thread, another loop

main.mjs ↗

```
import { Worker } from "node:worker_threads";
const t0 = Date.now(), ms = () => Date.now() - t0;
const w = new Worker(
  new URL("./fib.mjs", import.meta.url));
w.on("message", () => {
  console.log("fib(40) at", ms(), "ms");
  w.terminate();
});
w.postMessage(40);
setTimeout(() =>
  console.log("tick at", ms(), "ms"), 100);
```

fib.mjs ↗

```
import { parentPort } from "node:worker_threads";
const fib = n =>
  n < 2 ? n : fib(n - 1) + fib(n - 2);
parentPort.on("message", n =>
  parentPort.postMessage(fib(n)));
```

A **worker** is another thread with its own loop: its own stack, heap and queue.

The two threads share no object. `postMessage` sends a **copy**; it arrives as a function in the other thread's queue.

Run

recorded · node 25

tick at 104 ms

fib(40) at 563 ms

The tick is on time: the computation runs on the worker's thread. On one thread it came at 504 ms.

On each thread, Properties 1 to 4 hold; between threads, messages: session 8. The exception is a `SharedArrayBuffer`, memory both threads write; with it, data races return: session 7.

The repair

the read after the wait

```
async function deposit(n) {  
  const b = balance;  
  await approve(n);  
  balance = b + n;  
}  
async function repaired(n) {  
  await approve(n);  
  const b = balance;  
  balance = b + n;  
}
```

JavaScript ↗

Run

```
deposit first approval 100 ms: 120  
deposit first approval 200 ms: 110  
repaired first approval 100 ms: 130  
repaired first approval 200 ms: 130
```

The read moves after the `await`. The read and the write are then one stretch with no `await`: atomic, by Property 4.

Each deposit takes effect in that stretch. 130, whichever approval arrives first.

What was assumed before an `await` is re-established after it.